SPARK CATALYST & PHYSICAL EXECUTION PLAN GUIDE

Production Reference: 5-Stage Catalyst Pipeline, EXPLAIN Plan Anatomy, Join Hierarchies & Optimizer Rules

SPARK 3.5+ / 4.0 | CATALYST INTERNALS | PLAN ANATOMY | ISO 216 A4

5-Stage Catalyst Compiler Pipeline

Architecture

```
-- Catalyst 5-Stage Transformation Flow
SQL / DataFrame API
  ↳ 1. Unresolved Logical Plan (AST syntax validated, types unverified)
  ↳ 2. Analyzed Logical Plan (Catalog resolution: schemas, tables, functions)
  ↳ 3. Optimized Logical Plan (Rule-based optimization: pruning, pushdowns)
  ↳ 4. Physical Plans (CBO) (Cost-Based Optimizer selects best strategy)
  ↳ 5. Whole-Stage CodeGen (WSCG) (Janino compiles Java bytecode RDDs)
```

COMPILER MECHANICS & INVARIANTS

- **Analyzer Phase:** Uses `SessionCatalog` to verify column names and coerce mismatched data types.
- **Rule Execution:** Batches of idempotent tree-transformation rules run fixed-point iterations until tree stops mutating.
- **CBO Cost Evaluation:** Compares candidate physical plans using table row-counts, byte sizes, and column histograms.

Anatomy of EXPLAIN FORMATTED & Output

Plan Reading

```
-- Command to generate comprehensive operator tree
EXPLAIN FORMATTED SELECT c.id, sum(o.amt) FROM orders o
JOIN customers c ON o.cust_id = c.id
WHERE o.status = 'COMPLETE' GROUP BY c.id;

-- Tree Operator Output:
== Physical Plan ==
* (3) HashAggregate(keys=[id#12], functions=[sum(amt#15)])
+- Exchange hashpartitioning(id#12, 200), ENSURE_REQUIREMENTS, [id=#84]
   +- * (2) HashAggregate(keys=[id#12], functions=[partial_sum(amt#15)])
      +- * (2) BroadcastHashJoin [cust_id#14], [id#12], Inner, BuildRight
         :- * (2) Filter (isnotnull(status#16) AND (status#16 = COMPLETE))
         :  +- FileScan parquet [cust_id#14,amt#15,status#16]
         +- BroadcastExchange HashedRelationBroadcastMode [id=#78]
            +- * (1) Filter isnotnull(id#12)
               +- FileScan parquet [id#12]
```

NODE DECOUPLING & CONVENTIONS

- `*(N) Stage ID:` Denotes operator runs inside WholeStageCodegen pipeline `N` in a single JVM loop.
- `Exchange:` Hard stage boundary forcing network shuffle across cluster; cuts code generation boundaries.
- `+- Tree Hierarchy:` Read bottom-to-top from leaf scans to root sink; colons `:-` mark build vs probe branches.

Join Strategy Selection Hierarchy

Performance

```
-- Spark Join Evaluation Order (Descending Preference)
1. BroadcastHashJoin (BHJ):
   Equi-join & size < spark.sql.autoBroadcastJoinThreshold (10MB). Zero shuffle.
2. ShuffleHashJoin (SHJ):
   Equi-join, 1 side significantly smaller than other, build fits in executor memory.
3. SortMergeJoin (SMJ):
   Default for large equi-joins. Requires both sides partitioned & sorted on join keys.
4. BroadcastNestedLoopJoin (BNLJ):
   CRITICAL HAZARD: Triggered on non-equi joins (e.g. o.amt > t.limit). O(M×N) cartesia
n scan!
5. CartesianProduct:
   Explicit CROSS JOIN without conditions; catastrophic at scale.
```

JOIN RULES & ANTIDOTES

- **Join Hints:** Override optimizer with `/*+ BROADCAST(table) */` or `/*+ MERGE(table) */`.
- **BNLJ Avoidance:** Rewrite range joins with bucketed bounds or explode windowed intervals to restore equi-join hash mechanics.

Core Catalyst Optimizer Rules

Optimization

```
-- Key Transformation Rules in Optimizer Batch
PushDownPredicates:
  Moves Filter below Join and Aggregate down to storage FileScan.
ColumnPruning:
  Drops unused columns immediately after file read, minimizing memory buffers.
ConstantFolding:
  Evaluates 1 + 1 or compile-time constants into literal values at planning.
ReplaceDeduplicateWithAggregate:
  Rewrites expensive DISTINCT or window deduplication into streaming aggregates.
CollapseRepartition:
  Merges consecutive repartition() or coalesce() into single exchange.
```

PUSHEDFILTER VERIFICATION

- **Scan Check:** Verify `PushedFilters: [IsNotNull(id), EqualTo(status, 'COMPLETE')]` in leaf scan metadata.
- **Parquet Pruning:** Predicates must match file statistics (min/max) to skip entire row groups without disk reads.

Partitioning & Distribution Expressions

Shuffling

```
-- Spark Physical Data Distribution Operators
HashPartitioning(keys, numPartitions):
  Murmur3Hash(keys) % numPartitions. Standard for joins and aggregations.
RangePartitioning(order, numPartitions):
  Samples key distribution to partition data into sorted buckets (used by orderBy).
SinglePartition:
  DANGER: Collects entire distributed dataset onto 1 executor! Triggered by OVER ().
RoundRobinPartitioning(numPartitions):
  Uniformly scatters rows across tasks; disables join collocation optimizations.
```

SHUFFLE INVARIANT RULES

- **SinglePartition Triage:** Never write `ROW_NUMBER() OVER (ORDER BY col)` without a `PARTITION BY` key.
- **Co-partitioning:** When two tables have identical hash partitioning, SMJ skips exchange completely.

Whole-Stage CodeGen (WSCG) Mechanics

Compilation

```
-- Janino Java Code Compilation & Limits
spark.sql.codegen.wholeStage = true   -- Compiles tree into single Java loop
spark.sql.codegen.maxFields  = 100    -- Schema threshold before disabling WSCG

-- Symptom: Spark log warning "failed to compile: 64KB bytecode limit"
-- Root Cause: Wide tables (>150 cols) generate gigantic Java methods.
-- Remediation: Set config or project columns in stages:
spark.conf.set("spark.sql.codegen.hugeMethodLimit", 8000)
spark.conf.set("spark.sql.codegen.splitExpressions.threshold", 1024)
```

FALLBACK INDICATORS

- **Visual Missing Star:** If operator lacks `*(N)` in plan, code runs in Volcano iterator mode (10x slower).
- **Black Boxes:** Python UDFs and non-vectorized Hive SerDes break WSCG blocks into multiple stages.

SPARK PHYSICAL BOTTLENECKS & AQE TRIAGE

Production Reference: Adaptive Query Execution (AQE), Skew Joins, Exchange Bottlenecks & UI Metrics

AQE 3.5+ | TUNING & TRIAGE | UI METRICS | ISO 216 A4

Adaptive Query Execution (AQE) 3 Pillars

Runtime Dynamic

```
-- Enable Master Adaptive Query Execution
SET spark.sql.adaptive.enabled = true;

-- Pillar 1: Dynamic Partition Coalescing
SET spark.sql.adaptive.coalescePartitions.enabled = true;
SET spark.sql.adaptive.advisoryPartitionSizeInBytes = 134217728; -- 128MB

-- Pillar 2: Dynamic Switching to Broadcast Join
SET spark.sql.adaptive.autoBroadcastJoinThreshold = 20971520;   -- 20MB runtime

-- Pillar 3: Dynamic Skew Join Elimination
SET spark.sql.adaptive.skewJoin.enabled = true;
SET spark.sql.adaptive.skewJoin.skewedPartitionFactor = 5;
SET spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes = 268435456; -- 256MB
```

RUNTIME FEEDBACK MECHANICS

- **Stage Materialization:** AQE pauses between shuffle stages, reads shuffle partition sizes, and re-optimizes physical plan.
- **Initial Setting:** Set `spark.sql.shuffle.partitions = 2000`; AQE collapses empty partitions down to ideal 128MB chunks.

Exchange & Shuffle Bottleneck Elimination

Network I/O

```
-- Symptoms: Stage takes 90% of job time on Exchange operator
-- Diagnostic: Check Spark UI Shuffle Write Size vs Records Read

-- 1. Redundant Exchange Elimination:
-- BAD: Successive groupBy on different keys triggers 2 shuffles
df.groupBy("country").agg(...)
  .groupBy("region").agg(...)   -- 2 shuffles

-- GOOD: Single multi-dimensional rollup/cube:
df.rollup("region", "country").agg(...) -- 1 shuffle only!

-- 2. Bucketing Co-Location: Pre-shuffle on shared cluster key:
df.write.bucketBy(64, "cust_id").sortBy("cust_id").saveAsTable(...)
-- Joining two tables bucketed on same key generates zero Exchange nodes!
```

SHUFFLE WRITE TRIAGE

- **Spill to Disk:** Indicates shuffle buffer memory exhausted; tasks write intermediate blocks to local NVMe disk.
- **Remedy:** Increase `spark.sql.shuffle.partitions` or increase executor memory to fit hash buffers in heap.

Plan Anti-Patterns & Production Antidotes

Triage

```
-- Anti-Pattern 1: Non-Deterministic Python UDF Black Box
-- BROKEN: @udf disables Catalyst predicate pushdown & vectorization
@udf("string")
def clean(val): return val.strip().lower() if val else ""
-- FIX: Use native Spark SQL higher-order or expressions:
df.withColumn("clean", trim(lower(col("val"))))

-- Anti-Pattern 2: Uncorrelated Subquery Non-Pushdown
SELECT * FROM large_table
WHERE event_date = (SELECT max(d) FROM dates);
-- FIX: Join directly or collect scalar date in driver prior to query:
max_d = dates_df.select(max("d")).first()[0]
df.filter(col("event_date") == max_d)  -- Enables PartitionPruning!
```

QUERY PLAN SIGNALS

- `BatchEvalPython:` Py4J serialization boundary; replace with Pandas Vectorized UDF (`arrow`) or Spark SQL built-in.
- `SubqueryExec:` Subquery runs in sub-plan; evaluate whether dynamic partition pruning (DPP) is active.

Skew Join Mitigation & Two-Phase Salting

Data Skew

```
-- Manual Salting when AQE cannot split (e.g. non-Spark 3.x or custom ops)
from pyspark.sql.functions import expr, lit, concat

SALT_FACTOR = 16  -- Splits skewed key across 16 parallel tasks

-- 1. Salt the heavily skewed large table with random bucket [0..15]
skewed_df = large_df.withColumn(
    "salt", concat(col("join_key"), lit("_"), (expr("rand() * 16")).cast("int"))
)

-- 2. Explode the small dimension table across all 16 buckets
replicated_dim = dim_df.withColumn("salt_bucket", expr("sequence(0, 15)"))
.selectExpr("*", "explode(salt_bucket) as bucket")                    .withColumn
("salt", concat(col("join_key"), lit("_"), col("bucket")))

-- 3. Join on salted key (100% uniform partition distribution, zero stragglers!)
joined_df = skewed_df.join(replicated_dim, on="salt", how="inner")
```

PLAN VERIFICATION

- **AQE Plan Output:** Look for `SortMergeJoin(skew=true)` showing runtime split tasks.
- **Straggler Indicator:** In Spark UI, task 75th percentile runtime is 2s, but max task runtime is 25m (salting needed).

Spark UI SQL Tab & Metric Triage

Observability

```
-- Metric Formulas & Critical Thresholds
1. GC Time Ratio:
   GC Time / Task Duration > 10% → Severe Heap Exhaustion.
   Action: Increase executor memory or tune G1GC Region size (-XX:G1HeapRegionSize=32
m).
2. Shuffle Spill Ratio:
   Spill (Memory) / Spill (Disk) ≈ 2.5x to 4.0x (due to Snappy/LZ4 compression).
   Action: Partition count too small; increase spark.sql.shuffle.partitions.
3. Peak Execution Memory:
   Memory used by hash tables & sort buffers during stage execution.
4. Task Duration Distribution:
   Max / Median > 3.0x indicates structural data skew in stage.
```

DAG VISUALIZATION INSPECTION

- **Blue Rectangles:** WholeStageCodegen blocks; click to reveal underlying operator pipeline details.
- **Dotted Lines:** Shuffle exchange stage boundaries; identify network transfers and partition counts.

Memory & Execution Sizing Formulas

Sizing

```
-- Production Formula: Executor Total Memory Budget
Total Container RAM = ExecutorMemory + MemoryOverhead + OffHeapMemory

-- Sizing Calculation Rules:
1. Memory Overhead:
   max(384MB, 0.10 * executorMemory) (Required by YARN / K8s).
2. Unified Memory Pool (spark.memory.fraction = 0.60):
   Usable Memory = (executorMemory - 300MB) * 0.60
   ↳ Storage Pool (50%): DataFrame caching & broadcast variables.
   ↳ Execution Pool (50%): Shuffle, join, and sort buffers.
3. Production Flags for 32GB Executor:
   --executor-memory 28g --conf spark.executor.memoryOverhead=4096m
   --conf spark.memory.offHeap.enabled=true --conf spark.memory.offHeap.size=2g
```

OOM EXIT CODE 137 DEFENSE

- **Container Killed:** YARN/K8s kills container when off-heap memory + JVM exceeds container limit.
- **Remedy:** Always size `memoryOverhead` to $\ge 15\%$ when using PyArrow, MLlib, or heavy Python UDFs.